

Final question

Consider the infinite $x - y$ plane. Your task is to cover all the integer grid points on that plane (i.e. all the points (i, j) where $i \in \mathbb{Z}, j \in \mathbb{Z}$) with a repeating pattern of discs. A point on the edge of a disc is regarded as being covered. The discs may touch but not overlap. You may use discs of up to three different sizes. The objective is to use as few discs as possible, i.e. to maximise the mean number of points per disc, P . As a tie-break between solutions with the same P , the solution with the lowest area coverage Q (the fraction of the area of the plane that is covered by the discs) will be preferred.

You need to construct a **unit cell** for your solution. The unit cell must cover the plane using translations only; repeating identical copies of it must generate a set of whole discs that cover the grid points in the whole plane. If rotations of an original shape are required, attach the rotated shape to the original to form a unit cell that needs translations only.

For example, in Figure 1 below, the equilateral triangle ABD does not form a unit cell because a rotated copy is needed to tile the plane. However, the parallelogram (rhombus) $ABCD$ is a candidate unit cell.

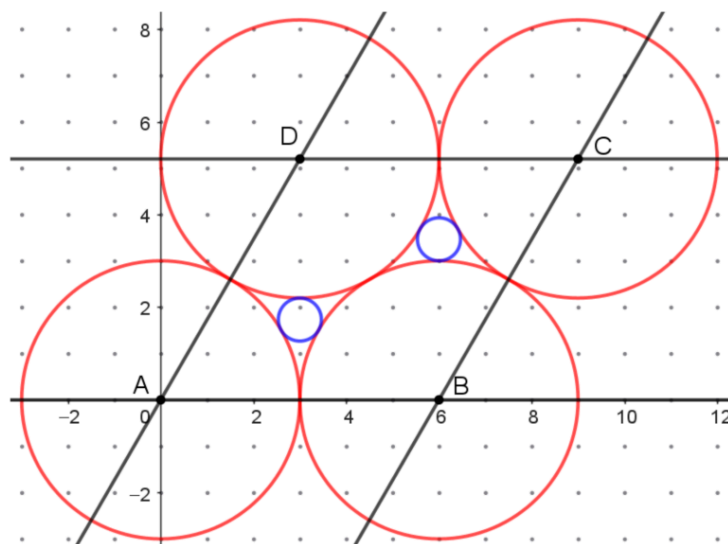


Figure 1 Candidate unit cell

However, there are a few issues with this 'solution':

- It uses only two sizes of disc (which is permitted, but is clearly not optimal).
- It does not cover the grid points $(3, 1)$, $(6, 4)$ or $(6, 5)$ within the proposed unit cell.
- Repeated translations of the proposed unit cell to cover the whole plane will not generate the required integer grid.

Once you have found your solution, you should submit as comma-separated data, rounded to 5 decimal places where necessary:

- the number of vertices of the unit cell
- for each vertex, the (x, y) coordinates (each vertex on a new line, proceeding around the polygon either clockwise or anti-clockwise)
- the number of discs that appear (at least partially) in each unit cell
- for each disc, the (x, y) coordinates of the centre of the disc, followed by its radius and the fraction of the disc that lies within the unit cell (new line for each disc; the discs can be entered in any order)
- P , the mean number of points per disc in the unit cell and hence for the infinite repeating pattern
- Q , the fraction of the area of the unit cell that is covered by the discs
- an image of the unit cell, showing the grid

Calculate P for the unit cell by adding up the fractions of discs included in the unit cell to get a total number of discs. Count points on an edge as $\frac{1}{2}$, points on a vertex that is shared by three unit cells as $\frac{1}{3}$, points on a 4-way vertex as $\frac{1}{4}$, etc. For example, the candidate unit cell $ABCD$ in the figure above contains:

- two quarter-points at A and B
- five half-points between A and B
- 30 other integer grid points
- one-sixth of a red disc centred at each of A and C
- one-third of a red disc centred at each of B and D
- two blue discs

Therefore, the mean number of points per disc in the proposed unit cell is $\frac{33}{3} = 11$.

The area of the rhombus is $18\sqrt{3}$. The large discs have radius 3 and the small discs radius $2\sqrt{3} - 3$. The unit cell contains the equivalent of one large disc and two small ones, so the fractional coverage is

$$Q = \frac{\pi \cdot 3^2 + 2 \cdot \pi (2\sqrt{3} - 3)^2}{18\sqrt{3}}$$

The data for the candidate unit cell and discs in the example above would be:

4

0,0

6,0

9,5.19615

3.5.19615

6

0,0,3,0.16667

6,0,3,0.33333

9,5.19615,3,0.16667

3,5.19615,3,0.33333

3,1.73205,0.46410,1

6,3.46410,0.46410,1

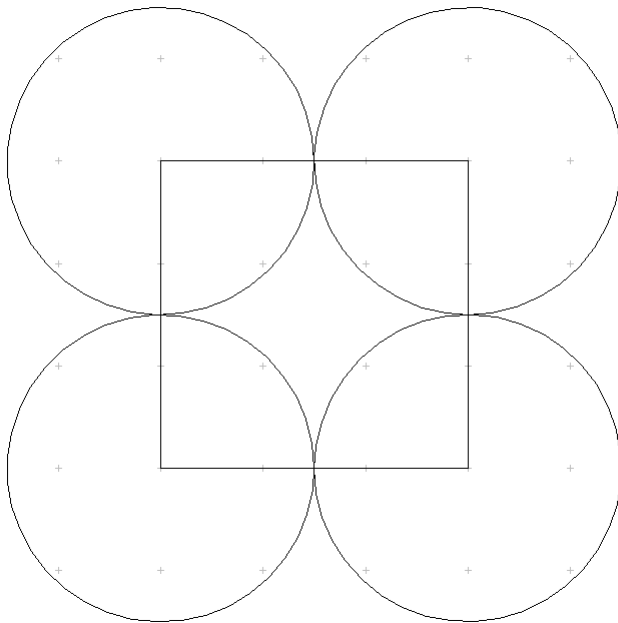
11.00000

0.95031

Working

Inspection Solutions

Solution 1



4

0,0

0,3

3,3

3,0

4

0,0,1.5,0.25

0,3,1.5,0.25

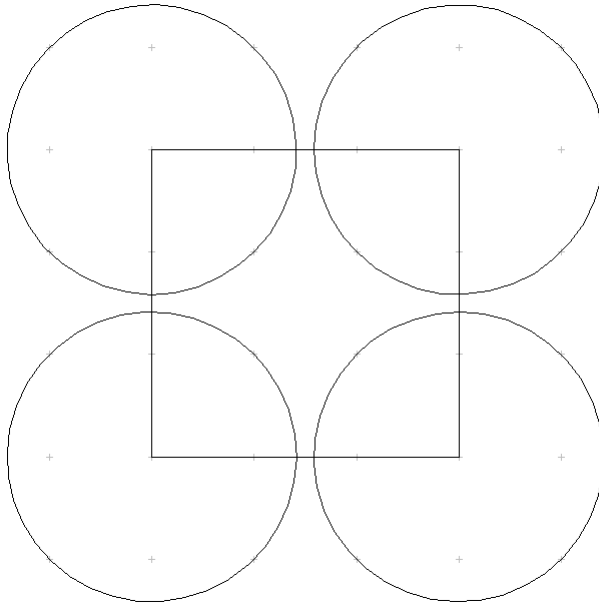
3,3,1.5,0.25

3,0,1.5,0.25

9

0.78540

Solution 2



```

4
0,0
0,3
3,3
3,0
4
0,0,1.41421,0.25
0,3,1.41421,0.25
3,3,1.41421,0.25
3,0,1.41421,0.25
9
0.69813

```

Code - Square Cell

```

In [7]: import matplotlib.pyplot as plt
from itertools import product, combinations
from math import sqrt, pi

iterations = 10
initialSideLength = 3

def distanceBetweenPoints(p1,p2):
    return sqrt((p2[0]-p1[0])**2 + (p2[1]-p1[1])**2)

def centerOfPoints(p1,p2):
    return ((p1[0]+p2[0])/2,(p1[1]+p2[1])/2)

def checkGridPoints(gridPoints,circles):
    covered = []
    for point in gridPoints:
        covered.append(any(distanceBetweenPoints(circle["Center"],point) <= circle["Radius"] for circle in circles))
    return covered

def addVectors(v1,v2):
    return (v1[0]+v2[0],v1[1]+v2[1])

def takeVectors(v1,v2):
    return (v2[0]-v1[0],v2[1]-v1[1])

ps = []
printStrings = []
correctCount = 0

for iteration in range(0,iterations*2,2): # Skip every other because it doesn't leave enough space between circles
    sideLength = iteration+initialSideLength
    halfSide = sideLength/2
    halfDiagonal = distanceBetweenPoints((0,0),(halfSide,halfSide))

    gridPoints = list(product(range(sideLength + 1),repeat=2))
    gridX, gridY = zip(*gridPoints)

    xs = [0,sideLength,sideLength,0]
    ys = [0,0,sideLength,sideLength]

    corners = list(zip(xs,ys))
    circles = []

    distances = []
    for point in gridPoints:
        distanceFrom0 = distanceBetweenPoints((0,0),point)
        if distanceFrom0 <= halfSide:
            distances.append(distanceFrom0)
    mainRadius = max(distances)

    for corner in corners:
        circles.append({"Center":corner,"Radius":mainRadius})

    gridPointsCovered = checkGridPoints(gridPoints,circles)

    if False in gridPointsCovered:
        center = (halfSide,halfSide)
        maximumDistances = sorted(list(set([distanceBetweenPoints(center,gridPoints[i]) for i in range(len(gridPoints)) if not gridPointsCovered[i]]),reverse=True))
        possibleMaximumDistances = [i for i in maximumDistances if halfDiagonal > i + mainRadius]

        circles.append({"Center":center,"Radius":possibleMaximumDistances[0]})

    gridPointsCovered = checkGridPoints(gridPoints,circles)

```

```

if False in gridPointsCovered:
    remainingPoints = [point for point in gridPoints if not gridPointsCovered[gridPoints.index(point)]]
    remainingPoints.sort(key=lambda x: x[1], reverse=True)
    topPoints = remainingPoints[0]
    for i in range(len(remainingPoints)):
        for point in remainingPoints:
            if any(distanceBetweenPoints(point, groupPoint) == 1 for groupPoint in topPoints):
                topPoints.add(point)

    maximumDistance = -1
    maximumCenter = None
    for points in combinations(topPoints, r=2):
        currentDistance = distanceBetweenPoints(*points) / 2 # div 2 To make radius
        if currentDistance > maximumDistance:
            maximumDistance = currentDistance
            maximumCenter = centerOfPoints(*points)
    offsetFromCenter = takeVectors(center, maximumCenter)

    if not any(distanceBetweenPoints(circle["Center"], maximumCenter) <= maximumDistance + circle["Radius"] for circle in circles):
        circles.append({"Center": (addVectors(center, offsetFromCenter)), "Radius": maximumDistance})
        circles.append({"Center": (addVectors(center, (offsetFromCenter[0], -offsetFromCenter[1]))), "Radius": maximumDistance})
        circles.append({"Center": (addVectors(center, (offsetFromCenter[1], offsetFromCenter[0]))), "Radius": maximumDistance})
        circles.append({"Center": (addVectors(center, (-offsetFromCenter[1], offsetFromCenter[0]))), "Radius": maximumDistance})

    gridPointsCovered = checkGridPoints(gridPoints, circles)

fig, ax = plt.subplots()
ax.set_aspect('equal', adjustable='box')
ax.set_xlim(-2, sideLength+2)
ax.set_ylim(-2, sideLength+2)
ax.set_xticks([-2, -1] + [i for i in range(sideLength+2)])
ax.set_yticks([-2, -1] + [i for i in range(sideLength+2)])
plt.xticks(fontsize=8)
plt.yticks(fontsize=8)
ax.plot(xs+[0], ys+[0])
# ax.plot(gridX, gridY, marker="+", markersize=5, linestyle="", color="#aaa")
ax.set_axisbelow(True)
ax.grid(True,)
for circle in circles:
    tempCircle = plt.Circle(circle["Center"], circle["Radius"], fill=False)
    ax.add_patch(tempCircle)

if False not in gridPointsCovered:
    ax.set_title(f"Correct, Index: {correctCount}", color="#00aa00")
    fig.savefig(f'./S3Plots/Square Plot - {correctCount} - Correct.png', dpi=96)
    printStr = f"{len(corners)}\n"

    for corner in corners:
        printStr += f"{{corner[0]}},{{corner[1]}}\n"

    printStr += f"{len(circles)}\n"
    qSum = 0
    for circle in circles:
        if circle["Radius"] == mainRadius:
            prop = 1/4
        else:
            prop = 1
        printStr += f"{{circle[\"Center\"]}}[0]},{{circle[\"Center\"]}}[1]},{{round(circle[\"Radius\"],5)}},{{prop}}\n"
        qSum += prop*pi*circle["Radius"]**2

    p = ((sideLength-1)**2 + (sideLength-1)*2 + 1)/(1 + len(circles)-4)
    q = qSum/sideLength**2

    printStr += f"{{round(p,5)}}\n{{round(q,5)}}"

    ps.append(p)
    printStrings.append(printStr)
else:
    ax.set_title(f"Incorrect, Index: {correctCount}", color="#ff0000")
    fig.savefig(f'./S3Plots/Square Plot - {correctCount} - Incorrect.png', dpi=96)
    ps.append(0)
    printStrings.append(None)
plt.show()

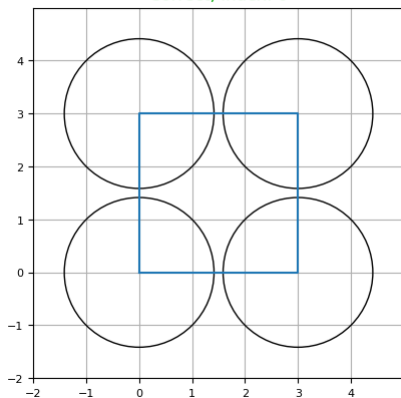
correctCount += 1

output = list(zip(ps, printStrings))
outputSorted = sorted(output, key=lambda x: x[0], reverse=True)

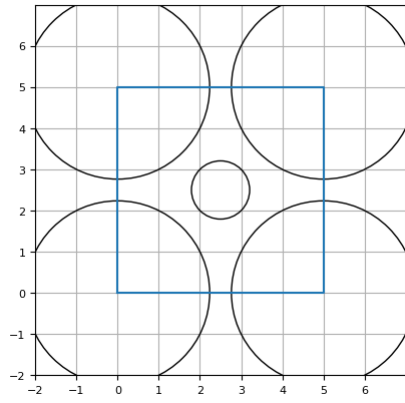
print(outputSorted[0][1])
print("----")
print(f"Graph Index: {output.index(outputSorted[0])}")

```

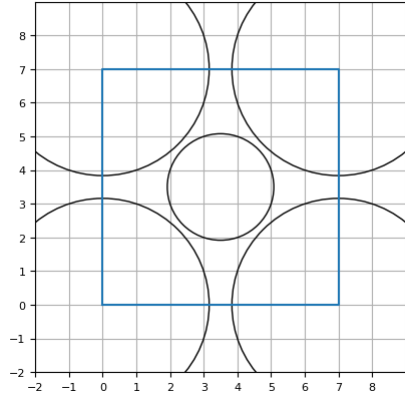
Correct, Index: 0



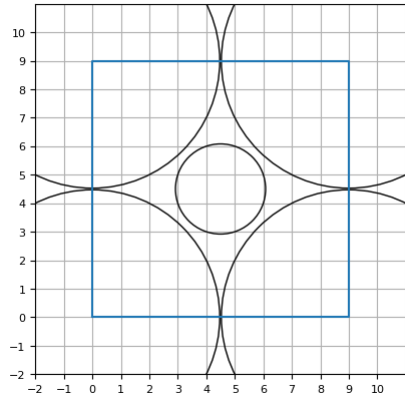
Correct, Index: 1



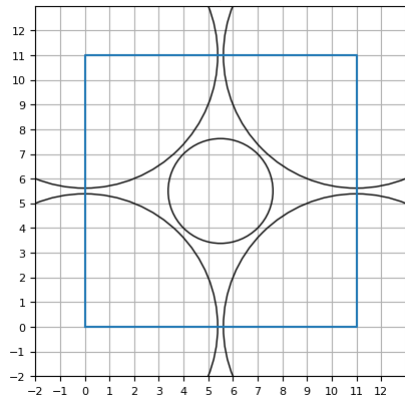
Correct, Index: 2



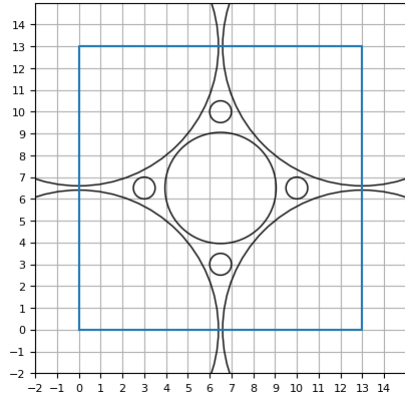
Correct, Index: 3



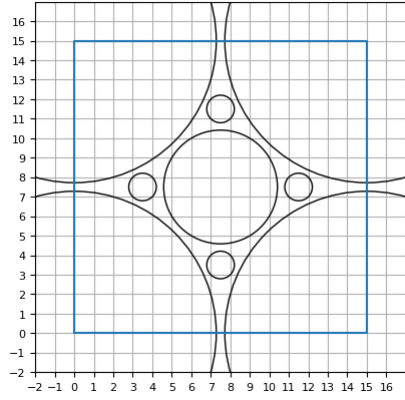
Incorrect, Index: 4



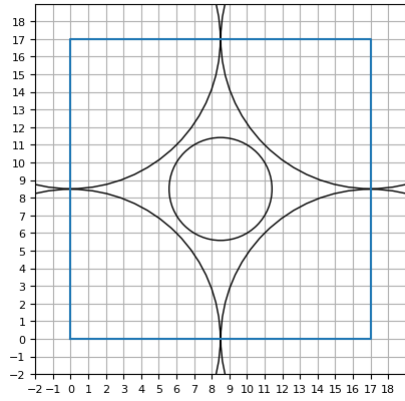
Correct, Index: 5



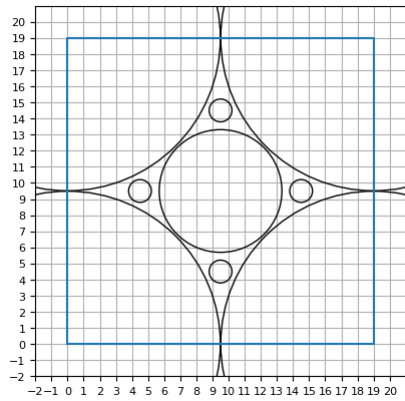
Correct, Index: 6

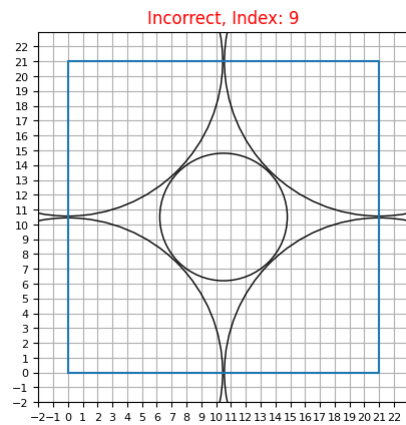


Incorrect, Index: 7



Correct, Index: 8





```
4
0,0
19,0
19,19
0,19
9
0,0,9.48683,0.25
19,0,9.48683,0.25
19,19,9.48683,0.25
0,19,9.48683,0.25
9.5,9.5,3.80789,1
9.5,14.5,0.70711,1
9.5,4.5,0.70711,1
14.5,9.5,0.70711,1
4.5,9.5,0.70711,1
60.16667
0.92681
---
```

Graph Index: 8

Code - Offset Square Cell

```
In [8]: import matplotlib.pyplot as plt
from itertools import product, combinations
from math import sqrt, pi

iterations = 10
initialSideLength = 2

def distanceBetweenPoints(p1,p2):
    return sqrt((p2[0]-p1[0])**2 + (p2[1]-p1[1])**2)

def centerOfPoints(p1,p2):
    return ((p1[0]+p2[0])/2,(p1[1]+p2[1])/2)

def checkGridPoints(gridPoints,circles):
    covered = []
    for point in gridPoints:
        covered.append(any(distanceBetweenPoints(circle["Center"],point) <= circle["Radius"] for circle in circles))
    return covered

def addVectors(v1,v2):
    return (v1[0]+v2[0],v1[1]+v2[1])

def takeVectors(v1,v2):
    return (v2[0]-v1[0],v2[1]-v1[1])

ps = []
printStrings = []
correctCount = 0

for iteration in range(0,iterations*2,2): # Skip every other because it doesn't leave enough space between circles
    sideLength = iteration+initialSideLength
    minSide = 0.5
    maxSide = 0.5 + sideLength
    halfSide = sideLength/2
    midPoint = halfSide + 0.5
    halfDiagonal = distanceBetweenPoints((minSide,minSide),(midPoint , midPoint))

    gridPoints = list(product(range(1,sideLength + 2),repeat=2))
    gridX, gridY = zip(*gridPoints)

    xs = [minSide,maxSide,maxSide,minSide]
    ys = [minSide,minSide,maxSide,maxSide]

    corners = list(zip(xs,ys))
    circles = []

    distances = []
    for point in gridPoints:
        distanceFromMin = distanceBetweenPoints((minSide,minSide),point)
        if distanceFromMin <= sideLength/2:
            distances.append(distanceFromMin)
        mainRadius = max(distances)

    for corner in corners:
        circles.append({"Center":corner,"Radius":mainRadius})

    gridPointsCovered = checkGridPoints(gridPoints,circles)

    if False in gridPointsCovered:
        center = (midPoint ,midPoint)
        maximumDistances = sorted(list(set([distanceBetweenPoints(center,gridPoints[i]) for i in range(len(gridPoints)) if not gridPointsCovered[i]]),reverse=True))
        possibleMaximumDistances = [i for i in maximumDistances if halfDiagonal > i + mainRadius]

        circles.append({"Center":center,"Radius":possibleMaximumDistances[0]})

    gridPointsCovered = checkGridPoints(gridPoints,circles)

    if False in gridPointsCovered:
        remainingPoints = [point for point in gridPoints if not gridPointsCovered[gridPoints.index(point)]]
        remainingPoints.sort(key=lambda x: x[1],reverse=True)
```



```

topPoints = {remainingPoints[0]}
for i in range(len(remainingPoints)):
    for point in remainingPoints:
        if any(distanceBetweenPoints(point,groupPoint) == 1 for groupPoint in topPoints):
            topPoints.add(point)

maximumDistance = -1
maximumCenter = None
for points in combinations(topPoints,n=2):
    currentDistance = distanceBetweenPoints(*points) / 2 # div 2 To make radius
    if currentDistance > maximumDistance:
        maximumDistance = currentDistance
        maximumCenter = centerOfPoints(*points)
offsetFromCenter = takeVectors(center,maximumCenter)

if not any(distanceBetweenPoints(circle["Center"],maximumCenter) <= maximumDistance + circle["Radius"] for circle in circles):
    circles.append({"Center":(addVectors(center,offsetFromCenter)), "Radius":maximumDistance})
    circles.append({"Center":(addVectors(center,(offsetFromCenter[0],-offsetFromCenter[1]))), "Radius":maximumDistance})
    circles.append({"Center":(addVectors(center,(offsetFromCenter[1],offsetFromCenter[0]))), "Radius":maximumDistance})
    circles.append({"Center":(addVectors(center,(-offsetFromCenter[1],offsetFromCenter[0]))), "Radius":maximumDistance})

gridPointsCovered = checkGridPoints(gridPoints,circles)

fig,ax = plt.subplots()
ax.set_aspect('equal',adjustable='box')
ax.set_xlim(-2,sideLength+2)
ax.set_ylim(-2,sideLength+2)
ax.set_xticks([-2,-1]+[i for i in range(sideLength+2)])
ax.set_yticks([-2,-1]+[i for i in range(sideLength+2)])
plt.xticks(fontsize=8)
plt.yticks(fontsize=8)
ax.plot(xs+[minSide],ys+[minSide])
# ax.plot(gridX,gridY,marker="+",markersize=5,linestyle="",color="#aaa")
ax.set_axisbelow(True)
ax.grid(True,)
for circle in circles:
    tempCircle = plt.Circle(circle["Center"],circle["Radius"],fill=False)
    ax.add_patch(tempCircle)

if False not in gridPointsCovered:
    ax.set_title(f"Correct, Index: {correctCount}",color="#00aa00")
    fig.savefig(f'./S3Plots/Offset Square Plot - {correctCount} - Correct.png',dpi=96)
    printStr = f"{len(corners)}\n"

    for corner in corners:
        printStr += f"{corner[0]},{corner[1]}\n"

    printStr += f"{len(circles)}\n"
    qSum = 0
    for circle in circles:
        if circle["Radius"] == mainRadius:
            prop = 1/4
        else:
            prop = 1
        printStr += f"{circle["Center"][0]},{circle["Center"][1]},{round(circle["Radius"],5)},{prop}\n"
        qSum += prop*pi*circle["Radius"]**2

    p = ((sideLength-1)**2 + (sideLength-1)*2 + 1)/(1 + len(circles)-4)
    q = qSum/sideLength**2

    printStr += f"{round(p,5)}\n{round(q,5)}"

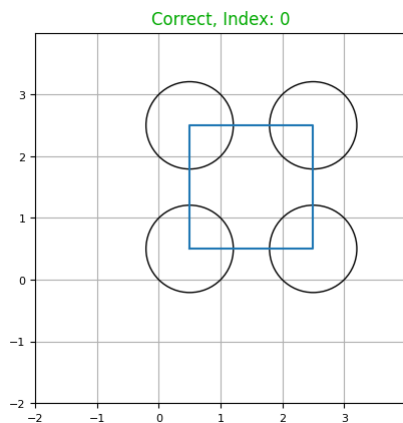
    ps.append(p)
    printStrings.append(printStr)
else:
    ax.set_title(f"Incorrect, Index: {correctCount}",color="#ff0000")
    fig.savefig(f'./S3Plots/Offset Square Plot - {correctCount} - Incorrect.png',dpi=96)
    ps.append(0)
    printStrings.append(None)
plt.show()

correctCount += 1

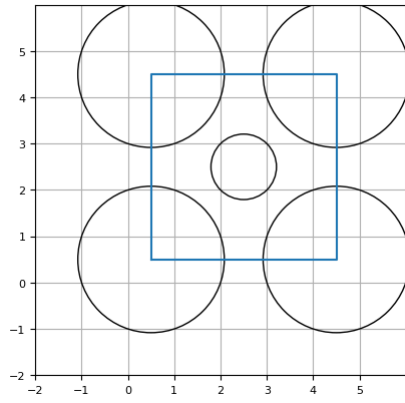
output = list(zip(ps,printStrings))
outputSorted = sorted(output,key= lambda x: x[0],reverse=True)

print(outputSorted[0][1])
print("----")
print(f"Graph Index: {output.index(outputSorted[0])}")

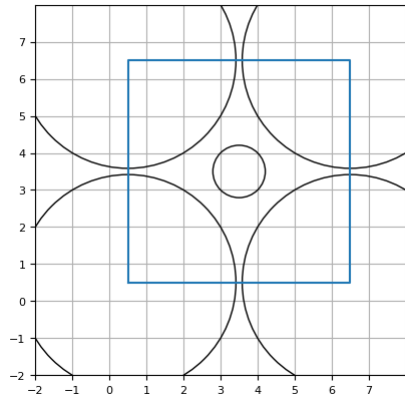
```



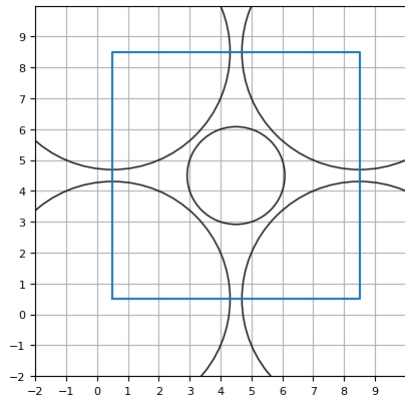
Correct, Index: 1



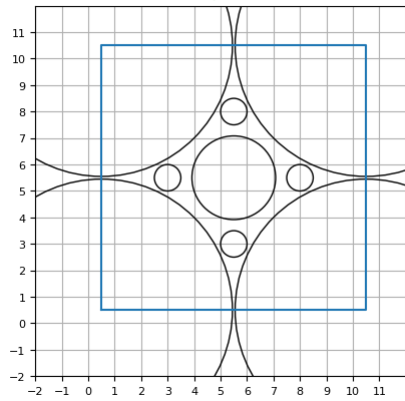
Correct, Index: 2



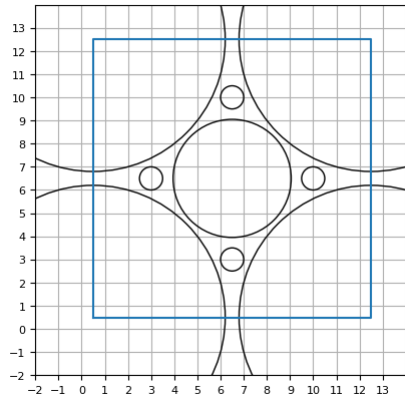
Correct, Index: 3



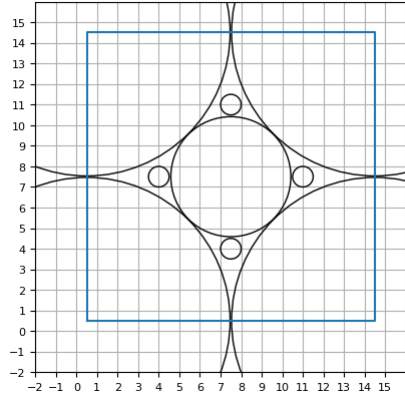
Correct, Index: 4



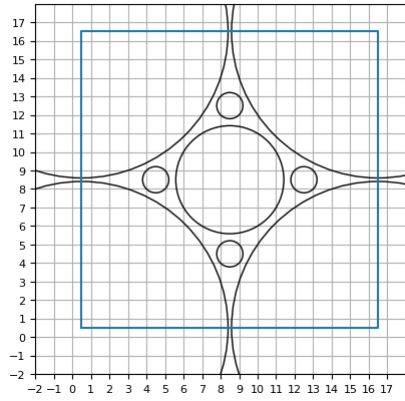
Correct, Index: 5



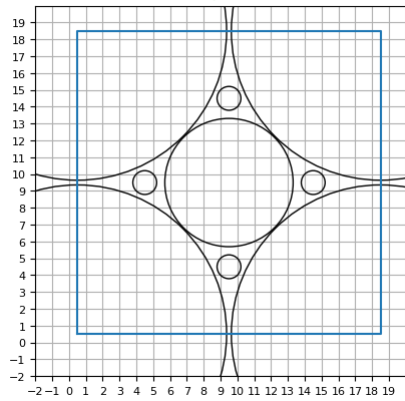
Correct, Index: 6

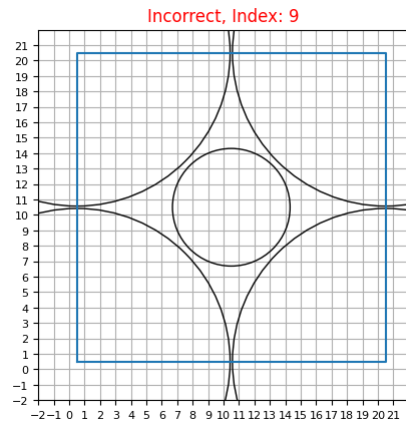


Correct, Index: 7



Correct, Index: 8





```
4
0.5,0.5
18.5,0.5
18.5,18.5
0.5,18.5
9
0.5,0.5,8.86002,0.25
18.5,0.5,8.86002,0.25
18.5,18.5,8.86002,0.25
0.5,18.5,8.86002,0.25
9.5,9.5,3.80789,1
9.5,14.5,0.70711,1
9.5,4.5,0.70711,1
14.5,9.5,0.70711,1
4.5,9.5,0.70711,1
54.0
0.92115
---
```

Graph Index: 8

Code - Square With More Discs

```
In [9]: import matplotlib.pyplot as plt
from itertools import product, combinations
from math import sqrt, pi

iterations = 10
initialSideLength = 3

def distanceBetweenPoints(p1,p2):
    return sqrt((p2[0]-p1[0])**2 + (p2[1]-p1[1])**2)

def centerOfPoints(p1,p2):
    return ((p1[0]+p2[0])/2,(p1[1]+p2[1])/2)

def checkGridPoints(gridPoints,circles):
    covered = []
    for point in gridPoints:
        covered.append(any(distanceBetweenPoints(circle["Center"],point) <= circle["Radius"] for circle in circles))
    return covered

def addVectors(v1,v2):
    return (v1[0]+v2[0],v1[1]+v2[1])

def takeVectors(v1,v2):
    return (v2[0]-v1[0],v2[1]-v1[1])

ps = []
printStrings = []
correctCount = 0

for iteration in range(0,iterations*2,2): # Skip every other because it doesn't Leave enough space between circles
    sideLength = iteration+initialSideLength
    halfSide = sideLength/2
    center = (halfSide,halfSide)
    halfDiagonal = distanceBetweenPoints((0,0),(halfSide,halfSide))

    gridPoints = list(product(range(sideLength + 1),repeat=2))
    gridX, gridY = zip(*gridPoints)

    xs = [0,sideLength,sideLength,0]
    ys = [0,0,sideLength,sideLength]

    corners = list(zip(xs,ys))
    circles = []

    distances = []
    for point in gridPoints:
        distanceFrom0 = distanceBetweenPoints((0,0),point)
        distanceFromCenter = distanceBetweenPoints(center,point)
        minDistance = min(distanceFrom0,distanceFromCenter)
        if minDistance <= halfDiagonal/2:
            distances.append(minDistance)
    mainRadius = max(distances)

    for corner in corners + [center]:
        circles.append({"Center":corner,"Radius":mainRadius})

    gridPointsCovered = checkGridPoints(gridPoints,circles)

    if False in gridPointsCovered:
        maximumDistances = sorted(list(set([distanceBetweenPoints((0,halfSide),gridPoints[i]) for i in range(len(gridPoints)) if not gridPointsCovered[i]]),reverse=True))
        possibleMaximumDistances = [i for i in maximumDistances if not any(distanceBetweenPoints(circle["Center"],(0,halfSide)) <= i + circle["Radius"] for circle in circles)]

        for edgeMid in [[0,halfSide],[halfSide,0],[halfSide,sideLength],[sideLength,halfSide]]:
            circles.append({"Center":edgeMid,"Radius":possibleMaximumDistances[0]})

    gridPointsCovered = checkGridPoints(gridPoints,circles)

    if False in gridPointsCovered:
        remainingPoints = [point for point in gridPoints if not gridPointsCovered[gridPoints.index(point)]]
        remainingPoints.sort(key=lambda x: x[1],reverse=True)
        topPoints = {remainingPoints[0]}
```

```

for i in range(len(remainingPoints)):
    for point in remainingPoints:
        if any(distanceBetweenPoints(point,groupPoint) == 1 for groupPoint in topPoints):
            topPoints.add(point)

maximumDistance = -1
maximumCenter = None
if len(topPoints) == 1:
    maximumCenter = topPoints.pop()
    maximumDistance = 0.00001
else:
    for points in combinations(topPoints,r=2):
        currentDistance = distanceBetweenPoints(*points) / 2 # div 2 To make radius
        if currentDistance > maximumDistance:
            maximumDistance = currentDistance
            maximumCenter = centerOfPoints(*points)
offsetFromCenter = takeVectors(center,maximumCenter)

if not any(distanceBetweenPoints(circle["Center"],maximumCenter) <= maximumDistance + circle["Radius"] for circle in circles):
    circles.append({"Center":(addVectors(center,( offsetFromCenter[0], offsetFromCenter[1])),,"Radius":maximumDistance)})
    circles.append({"Center":(addVectors(center,(-offsetFromCenter[0], offsetFromCenter[1])),,"Radius":maximumDistance)})
    circles.append({"Center":(addVectors(center,( offsetFromCenter[0],-offsetFromCenter[1])),,"Radius":maximumDistance)})
    circles.append({"Center":(addVectors(center,(-offsetFromCenter[0],-offsetFromCenter[1])),,"Radius":maximumDistance)})
    circles.append({"Center":(addVectors(center,( offsetFromCenter[1], offsetFromCenter[0])),,"Radius":maximumDistance)})
    circles.append({"Center":(addVectors(center,(-offsetFromCenter[1], offsetFromCenter[0])),,"Radius":maximumDistance)})
    circles.append({"Center":(addVectors(center,( offsetFromCenter[1],-offsetFromCenter[0])),,"Radius":maximumDistance)})
    circles.append({"Center":(addVectors(center,(-offsetFromCenter[1],-offsetFromCenter[0])),,"Radius":maximumDistance)})

gridPointsCovered = checkGridPoints(gridPoints,circles)

fig,ax = plt.subplots()
ax.set_aspect('equal',adjustable='box')
ax.set_xlim(-2,sideLength+2)
ax.set_ylim(-2,sideLength+2)
ax.set_xticks([-2,-1]+[i for i in range(sideLength+2)])
ax.set_yticks([-2,-1]+[i for i in range(sideLength+2)])
plt.xticks(fontsize=8)
plt.yticks(fontsize=8)
ax.plot(xs+[0],ys+[0])
# ax.plot(gridX,gridY,marker="+",markersize=5,linestyle="",color="#aaa")
ax.set_axisbelow(True)
ax.grid(True,)
for circle in circles:
    tempCircle = plt.Circle(circle["Center"],circle["Radius"],fill=False)
    ax.add_patch(tempCircle)

if False not in gridPointsCovered:
    ax.set_title(f"Correct, Index: {correctCount}",color="#00aa00")
    fig.savefig(f'./S3Plots/Square Plot More - {correctCount} - Correct.png',dpi=96)
    printStr = f"{len(corners)}\n"

    for corner in corners:
        printStr += f"{corner[0]},{corner[1]}\n"

    printStr += f"{len(circles)}\n"
    qSum = 0
    for circle in circles:
        if circle["Radius"] == mainRadius:
            prop = 1/4
        else:
            prop = 1
        printStr += f"{circle["Center"][0]},{circle["Center"][1]},{round(circle["Radius"],5)},{prop}\n"
        qSum += prop*pi*circle["Radius"]**2

    numberOfDiscs = 4 + len(circles) - 9 if (len(circles) > 9) else 2 + len(circles) - 5 if len(circles) > 5 else 2

    p = ((sideLength-1)**2 + (sideLength-1)*2 + 1)/numberOfDiscs
    q = qSum/sideLength**2

    printStr += f"{round(p,5)}\n{round(q,5)}"

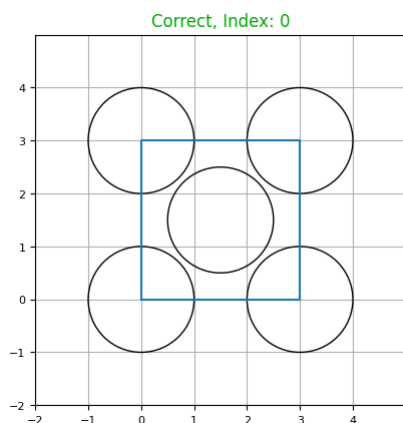
    ps.append(p)
    printStrings.append(printStr)
else:
    ax.set_title(f"Incorrect, Index: {correctCount}",color="#ff0000")
    fig.savefig(f'./S3Plots/Square Plot More - {correctCount} - Incorrect.png',dpi=96)
    ps.append(0)
    printStrings.append(None)
plt.show()

correctCount += 1

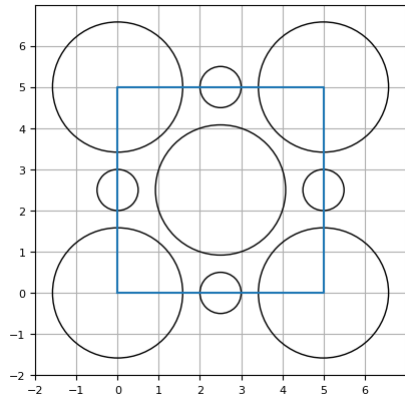
output = list(zip(ps,printStrings))
outputSorted = sorted(output,key= lambda x: x[0],reverse=True)

print(outputSorted[0][1])
print("----")
print(f"Graph Index: {output.index(outputSorted[0])}")

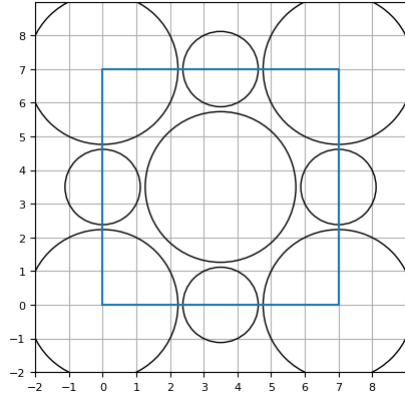
```



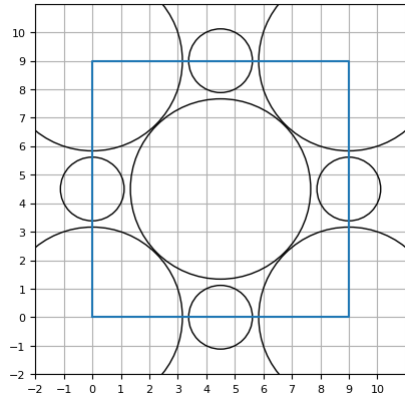
Correct, Index: 1



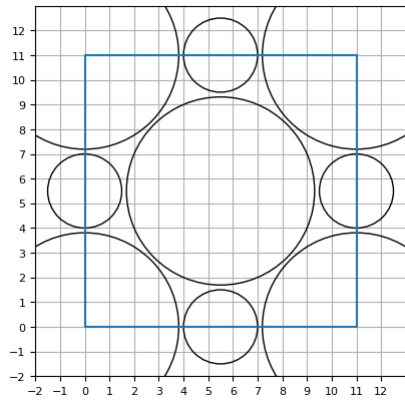
Correct, Index: 2



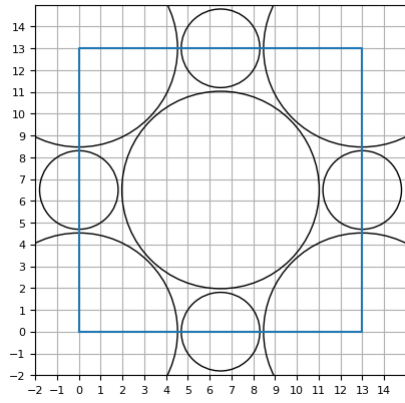
Correct, Index: 3



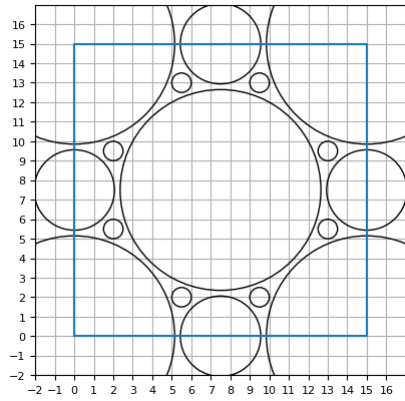
Correct, Index: 4



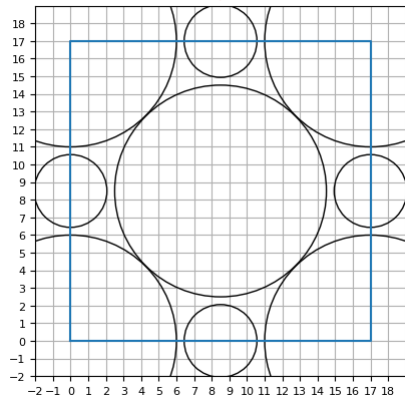
Correct, Index: 5



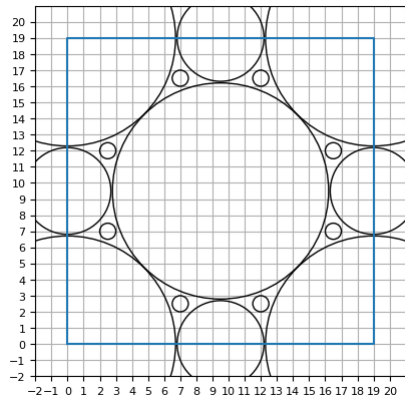
Correct, Index: 6



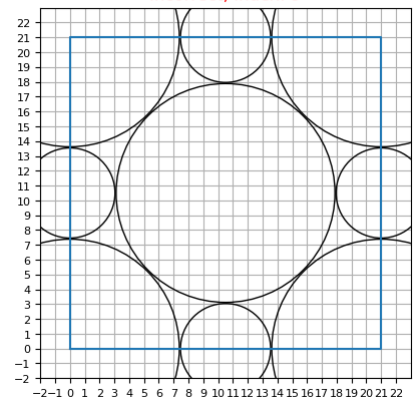
Incorrect, Index: 7



Correct, Index: 8



Incorrect, Index: 9



4
0,0
19,0
19,19
0,19
17
0,0,6.7082,0.25
19,0,6.7082,0.25
19,19,6.7082,0.25
0,19,6.7082,0.25
9.5,9.5,6.7082,0.25
0,9.5,2.69258,1
9.5,0,2.69258,1
9.5,19,2.69258,1
19,9.5,2.69258,1
7.0,16.5,0.5,1
12.0,16.5,0.5,1
7.0,2.5,0.5,1
12.0,2.5,0.5,1
16.5,7.0,0.5,1
2.5,7.0,0.5,1
16.5,12.0,0.5,1
2.5,12.0,0.5,1
30.08333
0.75929

Graph Index: 8